

محاسبي — الخطة التقنية (Tech Plan)

الوثيقة دي هي دليل التنفيذ. الـ Features والـ Billing موجودين في `PLAN.md` ، وهنا **إزاي هنبنيهم**: الـ Stack، الهيكل، القواعد، الداتايز، الـ APIs، المزامنة الأوفلاين، الـ Real time، الاختبارات، النشر، وخطة أول أسابيع يوم بيوم.

0. القرارات الأساسية (ملخص)

الموضوع	القرار
Backend	Laravel (آخر إصدار مستقر 12.x/13.x) + Modular Monolith — PHP 8.4 مقسم بالـ Domains
قاعدة البيانات	PostgreSQL 17
Real time	Laravel Echo + Laravel Reverb (WebSockets)
Queues	Laravel Horizon + Redis
Auth	Sanctum : كوكيز للوحة الويب، Tokens لأجهزة الـ POS وتطبيق المالك
الصلاحيات	<code>spatie/laravel-permission</code> بوضع Teams (الـ Team = المحل)
Frontend	Nuxt 4 + TypeScript + Nuxt UI (RTL) + Pinia — تطبيق واحد فيه اللوحة والكاشير
الكاشير أوفلاين	PWA + IndexedDB (Dexie) + Outbox Sync
تطبيق المالك	Flutter + Riverpod + Dio + go_router
لوحة إدارة المنصة (إحنا)	Filament داخل نفس مشروع Laravel
توثيق الـ API	Scramble (OpenAPI تلقائي) ← توليد TypeScript types للـ Nuxt
الفلوس	integer بالقرش + <code>brick/money</code>
IDs	UUIDv7 (بيتولد على الجهاز في الأوفلاين ومترتب زمنياً)
الاختبارات	Pest + Larastan + Pint / Vitest + Playwright
النشر	Cloudflare (DNS/R2) + Sentry + (Laravel Forge أو Coolify) Docker + VPS

ليه Modular Monolith؟ مشروع واحد، Deploy واحد، داتايز واحدة — أسرع حاجة لمطور واحد. والتقسيم بالـ Domains بيخلي الكود منظم وبسهل فصل أي جزء بعدين لو احتجنا.

1. هيكل الـ Repo (Monorepo)

```
muhasabi-egypt/
├── apps/
│   ├── api/           # Laravel - API + Reverb + Horizon + Filament (admin المنصة)
│   └── web/           # Nuxt 4 - الكاشير + المحل (PWA)
└──
```

```

├── owner/           # تطبيق المالك - Flutter
├── packages/
│   └── api-types/   # TypeScript types من OpenAPI (Scramble)
├── infra/
│   ├── docker/      # Dockerfiles + nginx + supervisor
│   └── compose.yml   # بيئة التطوير المحلية
├── docs/            # ADRs + ملاحظات تقنية
├── PLAN.md
├── TECH_PLAN.md
└── .github/workflows/ # CI

```

2. بيئة التطوير المحلية

فيه: `infra/compose.yml`

Port	الاستخدام	Service
5432	الداتابيز	pgsql (postgres:17)
6379	Queue + Cache + Sessions	redis (redis:7)
8025	استقبال الإيميلات في التطوير	mailpit
9000	S3 محلي (صور البطايق، الأجهزة، المرفقات)	minio
8000	FrankenPHP أو php artisan serve	api
8080	php artisan reverb:start	reverb
—	php artisan horizon	horizon
3000	pnpm dev (Nuxt)	web

أو ببساطة **Laravel Sail** للـ API، والـ Nuxt شغال على الجهاز مباشرة.

3. هيكل الـ Backend (Laravel)

3.1 التقسيم بالـ Domains

```

apps/api/app/
├── Domains/
│   ├── Identity/      # Tenants, Branches, Users, Roles, Devices, PIN login
│   └── Catalog/       # Categories, Brands, DeviceModels, Products, Variants,
Compatibility
│   └── Inventory/     # StockLevels, StockMovements, StockLots, SerialItems,
Stocktake, Transfers
│   ├── Sales/        # Sales, SaleItems, Payments, Returns, Holds
│   ├── Customers/    # Customers, CustomerLedger, Installments
│   ├── Suppliers/    # Suppliers, Purchases, SupplierLedger
│   ├── SupplierReturns/ # سلة المرتجعات حسب المصدر
│   ├── Imports/     # ImportContacts, Shipments, Costs, Payments, Claims
│   └── Repairs/      # Tickets, Faults, IntakeChecklist, StatusLogs, Parts

```

UsedDevices/	# شراء وبيع المستعمل
Services/	# كروت الشحن + تحويلات المحافظ
Cash/	# Shifts, CashMovements, Expenses
Messaging/	# MessageTemplates, MessageLogs, wa.me links
Billing/	# Plans, Subscriptions, Invoices, Payments, Gateways
Reports/	# Query classes للتقارير
Sync/	# Push/Pull للأجهزة الأوفلاين
Support/	# Money, Tenancy, Phone, Numbering, BaseModel ...

جوه كل Domain:

Domains/Sales/	
Models/	Sale.php, SaleItem.php, SalePayment.php
Enums/	SaleStatus.php, PaymentMethod.php
Actions/	CompleteSaleAction.php, VoidSaleAction.php, ReturnSaleItemsAction.php
Data/	SaleData.php (DTOs)
Events/	SaleCompleted.php
Listeners/	
Policies/	SalePolicy.php
Http/	
Controllers/	SaleController.php
Requests/	StoreSaleRequest.php
Resources/	SaleResource.php
routes.php	# بيتسجل من ServiceProvider

3.2 قواعد الكود (إجبارية)

- **Controllers رفيعة:** Validation في API Resource ← Action ← Form Request. مفيش Business logic في الـ Controller.
- **Action لكل عملية** (handle() واحدة), و Service لما عمليات كثير تشارك نفس الـ state.
- كل حاجة Enums: **typed** بدل الـ return types, properties, readonly, strings.
- **كل عملية بتلمس فلوس أو مخزون جوه** DB::transaction(), و الـ Jobs/Events بتتبع بعد الـ commit (afterCommit).
- مفيش Model::all() ولا get() من غير حد — Pagination دائماً.
- مفيش env() برا ملفات config/.
- Policies لكل Model — مفيش if (\$user->role == ...).
- Migrations قابلة للرجوع (down()).
- Pint + Larastan (Level 8) لازم يعدّوا قبل أي merge.

3.3 الحزم (Packages)

```
composer require laravel/sanctum laravel/reverb laravel/horizon laravel/pennant \
    spatie/laravel-permission spatie/laravel-query-builder spatie/laravel-activitylog \
    spatie/laravel-medialibrary brick/money propagandistas/laravel-phone \
```

```
dedoc/scramble filament/filament barryvdh/laravel-dompdf # أو spatie/laravel-pdf
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan \
laravel/pint laravel/telescope
```

الحزمة	ليه
spatie/laravel-permission	أدوار وصلاحيات لكل محل (Teams)
spatie/laravel-query-builder	فلتر وترتيب في الـ APIs بشكل مودد
spatie/laravel-activitylog	الـ Audit Log
spatie/laravel-medialibrary	الصور والمرفقات على S3/R2
brick/money	حسابات الفلوس من غير أخطاء float
propaganistas/laravel-phone	توحيد أرقام الموبايل المصرية (wa.me)
laravel/pennant	Feature flags حسب الباقية
dedoc/scramble	OpenAPI تلقائي من الكود
filament/filament	لوحة إدارة المنصة (المحلات، الاشتراكات، الموافقات اليدوية)

4. Multi-tenancy

قاعدة واحدة + `tenant_id` في كل جدول بيانات.

```
// app/Support/Tenancy/BelongsToTenant.php
trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope(new TenantScope);

        static::creating(function (Model $model): void {
            $model->tenant_id ??= app(CurrentTenant::class)->id();
        });
    }

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }
}
```

- `CurrentTenant` يحدد من المستخدم المسجل (`ResolveTenant Middleware`)، والفرع من `X-Branch-Id` Header (ومتأكد من أن المستخدم له صلاحية عليه).
- **خط دفاع ثاني: Row Level Security** Postgres على الجداول الحساسة (`SET app.tenant_id` في أول كل request). لو حد نسي الـ `scope`، الداتايز نفسها ترفض.

- **اختبار إجباري** (Pest Arch + Feature): كل Model في Domains/*/Models لازم يستخدم BelongsToTenant (ما عدا الاستثناءات المعروفة)، وتست إن مستخدم محل A مش بيشوف أي بيانات لمحل B.
- ال Queued Jobs بتشيل tenant_id ويتعمل restore لل context قبل ما تشتغل.
- spatie/laravel-permission بـ teams = true و team_id = tenant_id

5. تصميم قاعدة البيانات

5.1 قواعد عامة

- ال Primary key: uuid (UUIdv7) لكل الجداول اللي ممكن تتعمل أوفلاين (sales, repairs, customers, cash), و bigint للجداول الداخلية (lookup).
- كل جدول: tenant_id, created_at, updated_at, deleted_at (Soft delete) للكيانات الأساسية.
- الفلوس: bigint بالقرش — عمود اسمه *_piasters أو Cast اسمه MoneyCast.
- الكميات: integer (القطع). مفيش كسور في المخزون.
- Index على (tenant_id, ...) في كل حاجة بيتعمل عليها بحث.
- البحث: pg_trgm + Index على اسم الصنف والكود والباركود (بحث عربي/إنجليزي جزئي سريع).

5.2 الجداول الأساسية (المرحلة الأولى)

tenants	id, name, slug, phone, tax_id, settings(jsonb), created_at	
branches	id, tenant_id, name, address, phone, invoice_prefix, is_active	
users	id, tenant_id, name, phone, email, password, pin_hash, is_owner	
devices	id(uuid), tenant_id, branch_id, name, platform, last_seen_at,	
device_code		
categories	id, tenant_id, parent_id, name, type(enum:	
accessory part device service)		
brands	id, tenant_id, name	
device_models	id, tenant_id, brand_id, name, release_year	كتالوج مشترك + --
خاص		
products	id, tenant_id, category_id, brand_id, name, sku, track_serial(bool),	
is_active		
product_variants	id, tenant_id, product_id, name, barcode, quality_grade(enum),	
	price_retail, price_wholesale, price_technician, min_stock	
product_compatibility	variant_id, device_model_id	-- PK مرّكب
stock_levels	tenant_id, branch_id, variant_id, qty, avg_cost	-- PK (branch_id,
variant_id)		
stock_lots	id, tenant_id, branch_id, variant_id, source_type(enum:	
supplier import_shipment opening customer_return),		
	source_id, source_ref_id, unit_cost, qty_in, qty_remaining, received_at	
stock_movements	id, tenant_id, branch_id, variant_id, lot_id, type(enum), qty(+/-),	
	unit_cost, ref_type, ref_id, user_id, created_at	-- Append-only
serial_items	id, tenant_id, variant_id, branch_id, lot_id, serial, status(enum)	
customers	id(uuid), tenant_id, name, phone, phone_e164, notes, credit_limit,	
balance		
customer_ledger	id, tenant_id, customer_id, type, amount, ref_type, ref_id, created_at	

```

suppliers          id, tenant_id, name, phone, type(enum: local|shop), balance
supplier_ledger    id, tenant_id, supplier_id, type, amount, ref_type, ref_id
purchases / purchase_items
sales              id(uuid), tenant_id, branch_id, device_id, shift_id, number,
customer_id,
                  status, subtotal, discount, total, paid, due, cost_total, completed_at
sale_items         id, sale_id, variant_id, lot_id, serial_item_id, qty, unit_price,
discount, unit_cost
sale_payments      id, sale_id, method(enum), amount, reference
shifts             id(uuid), tenant_id, branch_id, user_id, opened_at, closed_at,
opening_cash, expected(jsonb), actual(jsonb)
cash_movements / expenses / expense_categories
activity_log       (spatie)

```

5.3 جداول الصيانة

```

fault_categories    id, tenant_id, name, sort -- شاشة، بوردة، سوكيت
شحن...
fault_types         id, tenant_id, category_id, name, default_labor_price, is_active,
sort
intake_checklist_items id, tenant_id, label, sort -- ... بيفتح؟ بي شحن؟
repair_tickets      id(uuid), tenant_id, branch_id, number, customer_id,
device_model_id, imei, color,
passcode_encrypted, accessories(jsonb), condition(jsonb),
intake_checks(jsonb),
status(enum), technician_id, received_at, received_by, expected_at,
delivered_at, labor_price, deposit, total, warranty_days, notes
repair_ticket_faults ticket_id, fault_type_id, kind(enum: reported|diagnosed), note
repair_status_logs  id, ticket_id, from_status, to_status, user_id, note, created_at
repair_parts        id, ticket_id, variant_id, lot_id, qty, unit_price, unit_cost

```

الأعطال الافتراضية (الجدول الي في `PLAN.md`) بتعمل **Seed** لكل محل جديد وقت التسجيل.

5.4 جداول الاستيراد ومرتجعات الموردين

```

import_contacts     id, tenant_id, type(enum: factory|agent|shipping|clearance), name,
country, phone, wechat, notes
import_shipments    id, tenant_id, contact_id, number, status(enum), ordered_at,
expected_at, arrived_at,
cost_allocation(enum: value|qty|weight), notes
import_shipment_items id, shipment_id, variant_id, qty_ordered, qty_received,
qty_damaged, unit_price, landed_unit_cost, weight
import_costs        id, shipment_id, type(enum:
shipping|customs|clearance|transport|commission|other), contact_id, amount
import_payments     id, tenant_id, contact_id, shipment_id(nullable), amount, method,
paid_at,
reference, receiver, original_amount_note, proof(media)
import_claims       id, shipment_id, type(enum: shortage|damaged), amount, status

supplier_returns    id, tenant_id, source_type, source_id, number, status(enum:
pending|sent|accepted|rejected), sent_at, settled_at
supplier_return_items id, return_id, variant_id, lot_id, serial_item_id, qty, unit_cost,
reason(enum), outcome(enum: credit|replace|refund|rejected)

```

- حساب كل جهة استيراد = Σ (تكلفة الشحنات + المصاريف المنسوبة ليها) - Σ الدفعات - Σ المطالبات المقبولة. بيتحسب في **View** أو Query class، مش عمود بيتعدّل بإيد.

5.5 جداول الرسائل والـ Billing

```
message_templates id, tenant_id, key, context(enum:
repair|sale|customer|supplier|import|general), name, body, is_active
message_logs id, tenant_id, template_id, user_id, recipient_phone, context_type,
context_id, rendered_body, opened_at
public_links id(token), tenant_id, linkable_type, linkable_id, expires_at --
لينكات الفواتير والإيصالات

plans, plan_features, subscriptions, subscription_items, billing_invoices,
billing_invoice_lines,
billing_payments, payment_methods, coupons, credit_ledger, usage_counters, billing_events
```

(التفاصيل في PLAN.md قسم 3.9 — الجداول دي **مش** عليها tenant_id scope الأوتوماتيكي لأنها بتاعة المنصة).

6. محرك المخزون (أهم جزء في الـ Business logic)

القاعدة: stock_movements هو مصدر الحقيقة (Append-only، مفيش update ولا delete). stock_levels و stock_lots.qty_remaining مجرد أرصدة بتتحدّث في **نفس الـ Transaction**.

```
final class CompleteSaleAction
{
    public function __construct(
        private readonly StockService $stock,
        private readonly NumberingService $numbering,
    ) {}

    public function handle(SaleData $data, User $cashier): Sale
    {
        return DB::transaction(function () use ($data, $cashier): Sale {
            // Idempotency: رجّعه زي ما هو UUID لو الـ
            if ($existing = Sale::find($data->id)) {
                return $existing;
            }

            $sale = Sale::create([...]);

            foreach ($data->items as $item) {
                // lockForUpdate على stock_levels + الـ Lots استهلاك الـ FIFO
                $consumed = $this->stock->consume(
                    branchId: $data->branchId,
                    variantId: $item->variantId,
                    qty: $item->qty,
                    ref: $sale,
                    serialId: $item->serialItemId,
                );
                $sale->items()->create([...$item->toArray(), 'lot_id' => $consumed->lotId,
                    'unit_cost' => $consumed->unitCost]);
            }
        });
    }
}
```

```

    $sale->payments()->createMany($data->payments);
    $this->postToCustomerLedgerIfCredit($sale);

    SaleCompleted::dispatch($sale); // ShouldBroadcast + afterCommit
    return $sale;
  });
}
}

```

- **التكلفة:** متوسط مرجّح (avg_cost) للتقارير، وكل بيعة بتأخذ unit_cost من ال Lot المستهلك (FIFO) — ده اللي بيخّلي المرتجع يعرف مصدره والمكسب دقيق.
- البيع بالسالب مسموح (خصوصاً في الأوفلاين) بس بيطلع تنبيه تسوية.
- **الاستيراد:** عند استلام الشحنة ← حساب landed_unit_cost لكل صنف ← إنشاء stock_lots بمصدر import_shipment
- **المرتجع من عميل:** يرجع لـ Lot بتاعه (سليم) أو يروح supplier_returns بمصدر ال Lot (معيّب).
- **التحويل بين الفروع:** Movement خروج من فرع + دخول لفرع بنفس ال Lot source (عشان المصدر ميضيعش).

7. الكاشير الأوفلاين والمزامنة

7.1 على الجهاز (Nuxt PWA)

- Service Worker : @vite-pwa/nuxt يكاش ال App shell فيفتح من غير نت.
- **Dexie (IndexedDB)** فيه نسخة من: الأصناف والأسعار والباركود، المخزون الحالي للفرع، العملاء، الإعدادات، القوالب.
- **Outbox:** كل عملية (بيعة، مرتجع، عميل جديد، حركة خزنة، تذكرة) بتسجل محلياً بـ UUIDv7 وتتحط في طابور.
- ترقيم الفواتير محلياً: {branch_prefix}-{device_code}-{seq} — مفيش تعارض بين الأجهزة.
- حالة الاشتراك متكاشة على الجهاز مع مهلة 7 أيام أوفلاين.

7.2 API

```

POST /api/v1/sync/push { device_id, operations: [ {id, type, payload, created_at}, ... ] }
← { results: [ {id, status: applied|duplicate|rejected, error?} ] }

GET /api/v1/sync/pull?since=<cursor>&entities=products,stock,customers,templates
← { changes: {...}, next_cursor }

```

- ال Push بيتنقّذ بالترتيب وكل عملية Idempotent على ال id.
- ال Pull بـ cursor على updated_at + id (Delta sync)، وأول مرة Full snapshot.
- التعارضات: السيرفر هو المرجع للأسعار والأصناف، والجهاز هو المرجع لعملياته (البيعة اللي حصلت حصلت).

- فشل عملية (مثلاً صنف اتمسح) ← تتعلم rejected وتظهر للمدير يحلها يدوي — عمرها ما تتمسح بصمت.

7.3 الترتيب في التنفيذ

1. **MVP:** الكاشير أونلاين بس يكتب في Dexie الأول (Local-first UI) وبيعت فوراً.
2. **Beta:** تفعيل الأوفلاين الكامل + الـ Outbox + شاشة "عمليات مستتية مزمنة".

8. الـ Real time (Reverb)

مين بيسمع	Events	Channel
لوحة الفرع، الكاشير	SaleCompleted , StockLow , RepairStatusChanged , ShiftClosed	private-tenant. {t}.branch.{b}
تطبيق المالك، لوحة المالك	ApprovalRequested (خصم كبير/مرتجع), DailySummaryReady , ملخص المبيعات	private-tenant. {t}.owner
شاشة الفنيين	RepairTicketCreated , RepairStatusChanged	private-tenant. {t}.repairs
المستخدم	إشعارات شخصية	private-user.{u}

- Authorization للـ Channels في routes/channels.php يتأكد من الـ tenant والفرع.
- Flutter . pusher-js + laravel-echo :Nuxt: dart_pusher_channels (بروتوكول Pusher اللي Reverb بيدعمه).
- الـ Events بتتبعث ShouldBroadcast + afterCommit عشان محدث يستلم حدث لعملية اترجعت.
- Payload صغير (IDs + أرقام أساسية)، والعمل يعمل refetch لو محتاج تفاصيل.

9. الطباعة والباركود

- **المرحلة 1:** طباعة من المتصفح بـ CSS مخصص لـ 80mm/58mm (@page { size: 80mm auto }) — بتشتغل مع أي طابعة حرارية متعرفة على الويندوز.
- **المرحلة 2: QZ Tray** (أو Print agent صغير) لطباعة ESC/POS مباشرة من غير نافذة الطباعة + فتح درج الكاش.
- لبيلات الباركود: JsBarcode + قالب طباعة لمقاسات الليبل الشائعة (مثلاً 25mm×38).
- قراءة الباركود: القارئ USB يشتغل كـ Listener ← Keyboard في الـ POS يفرّق بين الكتابة السريعة (سكانر) والكتابة العادية. + مسح بالكاميرا في الموبايل (@zxing/browser).
- الفاتورة A4 و PDF ← PDF: Blade template أو DomPDF أو Browsershot لدعم العربي الأحسن).

10. قوالب WhatsApp

```
final class RenderMessageAction
{
    public function handle(MessageTemplate $template, Model $context, User $user):
    RenderedMessage
    {
        $vars = $this->variablesFor($context); // {رقم_التذكرة}, {اسم_العميل},
        {المبلغ}...
        $body = strtr($template->body, $vars);
        $phone = phone($this->recipientPhone($context), 'EG')->formatE164(); // +2010...

        $log = MessageLog::create([...]); // سجل " افتحت
        return new RenderedMessage(
            body: $body,
            url: 'https://wa.me/'.ltrim($phone, '+').'?text='.rawurlencode($body),
        );
    }
}
```

- ال Frontend يعرض الرسالة للتعديل، وبعددين `window.open(url)`.
- لينك الفاتورة/الإيصال: `https://app.domain/r/{token}` (جدول `public_links` , token عشوائي 32 حرف، ومفيهوش بيانات حساسة).
- القوالب الافتراضية بتتعمل Seed لكل محل جديد.

11. ال Billing (تقنياً)

```
interface PaymentGateway
{
    public function createCheckout(BillingInvoice $invoice, PaymentMethodType $method):
    CheckoutSession;
    public function chargeSavedMethod(BillingInvoice $invoice, PaymentMethod $method):
    ChargeResult;
    public function verifyWebhook(Request $request): WebhookEvent; // HMAC
}
// Drivers: PaymobGateway, FawryGateway, ManualGateway (InstaPay/مندوب)
```

- Webhooks: `POST /webhooks/{gateway}` ← تحقق HMAC ← `billing_events` ← Unique على `gateway_ref` (Idempotent) ← Job يطبق الدفع.
- **Scheduler يومي**: `billing:generate-invoices` , `billing:charge-due` , `billing:transition-states` (restricted) ← `past_due` ← `suspended` , `billing:send-reminders`.
- **Pennant** + `EnsureSubscriptionAllows` Middleware لل `features` حسب الباقية، و `PlanLimits` للحدود (مستخدمين، فروع، أصناف).
- وضع RESTRICTED: القراءة والبيع مسموحين، الإنشاء في الكيانات المحددة ممنوع (403 برسالة واضحة + بانر).

- لوحة Filament: المحلات، الاشتراكات، الموافقة على تحويلات InstaPay (صورة الإيصال)، الكوبونات، المندوبين.

12. API Conventions

- Prefix: `/api/v1/...` — `Route::apiResource` للـ CRUD.
- الرد دائماً API Resource: `json { "data": {...}, "meta": {...} }`
- شكل الأخطاء موحّد (Exception handler): `{ "message": "الرصيد غير كافي", "code": "insufficient_credit", "errors": { "field": [...] } }`
- الأكواد: 201 إنشاء، 403 422 Validation صلاحيات/باقية، 404، 409 تعارض، 429 Rate limit.
- Pagination: Cursor للقوائم الكبيرة (المبيعات، الحركات)، و Page للإعدادات.
- فلترة وترتيب: `?filter[status]=ready&sort=-received_at&include=customer` (spatie/query-builder).
- Rate limiting على Login و PIN و Sync و Webhooks.
- `Accept-Language: ar` افتراضي — رسائل الـ Validation بالعربي.

13. الـ Frontend (Nuxt 4)

```
pnpm dlx nuxi@latest init apps/web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexe \
  laravel-echo pusher-js jsbarcode @zxing/browser echarts vue-echarts zod
pnpm add -D vitest @vue/test-utils @playwright/test openapi-typescript
```

```
apps/web/app/
├── pages/
│   ├── pos/                # اختصارات (Fullscreen، مستقل layout) الكاشير
│   ├── repairs/            # التذاكر + استلام جهاز
│   ├── inventory/ products/ purchases/ suppliers/ customers/
│   ├── imports/ supplier-returns/
│   └── cash/ reports/ settings/ billing/
├── layouts/ default.vue (Sidebar RTL) · pos.vue
├── composables/ useApi.ts · useAuth.ts · useBranch.ts · useScanner.ts · useWhatsApp.ts · useEcho.ts
├── stores/ (Pinia) auth · cart · settings · sync
├── offline/ db.ts (Dexie schema) · outbox.ts · sync.ts
└── utils/ money.ts (قرش ≠ جنيه) · phone.ts · print.ts
```

- **RTL أولًا:** `dir="rtl"` + خط **Cairo** أو IBM Plex Sans Arabic، والأرقام إنجليزي (أوضح في الكاشير) مع إعداد لتحويلها.
- Types من الـ API: `openapi-typescript` على الـ spec الي Scramble بيطلعها ← `packages/api-types`.
- Auth: Sanctum SPA cookies، للوحة، والـ POS بـ Device token + دخول الكاشير بـ PIN.

- أداء الكاشير: البحث في Dexie (مش في السيرفر)، هدف إضافة صف > 100ms.

14. تطبيق المالك (Flutter)

- `firebase_messaging` , `dart_pusher_channels` , `json_serializable` + `freezed` , `go_router` , `dio` , `flutter_riverpod` (Push), `mobile_scanner` , `fl_chart` (الجرد بالكاميرا).
- الشاشات: الملخص اللحظي، الموافقات، التذاكر، النواقص، الآجل، الاستيراد (الأرصدة والشحنات)، التقارير.
- Architecture: Feature-first (`features/dashboard` , `features/approvals` ...) + Repository لكل Feature.
- يتبني بعد ما ال API يستقر (المرحلة P1) — مش من أول يوم.

15. الأمان

- تشفير على مستوى الحقل (`cast encrypted`): الباسورد/النمط، الرقم القومي. صور البطايق في Bucket خاص بـ Signed URLs قصيرة العمر.
- 2FA للمالك (Fortify), PIN للكاشير مع قفل بعد 5 محاولات.
- إدارة الأجهزة: المالك يشوف الأجهزة المتصلة ويلغي أي Token.
- Audit Log لكل العمليات الحساسة (إلغاء، مرتجع، تعديل سعر، خصم فوق الحد، فتح الدرج، رؤية بيانات مشفرة).
- HTTPS, CSP headers فقط, Cookies `Secure` / `HttpOnly` / `SameSite=Lax`.
- `pnpm audit` g `composer audit` في ال CI.

16. الاختبارات و ال CI

الطبقة	الأداة	يغطي إيه
Arch	Pest Arch	DB, كل Model عليه <code>tenant trait</code> , مفيش (<code>env</code>)
Unit	Pest	Money, توزيع تكلفة الاستيراد, Proration, FIFO, تجهيز رقم الموبايل
Feature	Pest	كل Endpoint: نجاح + Validation + ملاحظات + عزل المحلات
Concurrency	Pest	بيعتين في نفس اللحظة لنفس الصنف ← المخزون صح
Frontend unit	Vitest	Cart, الحسابات، ال Outbox

الطبقة	الأداة	يغطي إيه
E2E	Playwright	رحلة: فتح وردية ← بيع ← مرتجع ← قفل وردية. واستلام جهاز صيانة ← تسليم. ووضع الأوفلاين (قطع النت وتكملة البيع)

GitHub Actions (على كل PR):

```
api: composer install → pint --test → larastan (level 8) → pest --parallel (Postgres service)
web: pnpm install → nuxi typecheck → eslint → vitest → nuxi build
e2e: (على main) docker compose up → playwright
```

17. النشر (DevOps)

البيئة	المكان
Local	Docker compose / Sail
Staging	VPS صغير (نفس إعدادات الـ Production) — كل merge على main
Production	4 vCPU / 8GB VPS (Hetzner/DigitalOcean) في البداية

- **Docker images**: api (FrankenPHP أو php-fpm + nginx), worker (Horizon), scheduler, reverb, web (Nuxt) — والإدارة بـ **Coolify** أو **Laravel Forge**.
- **Postgres**: على VPS منفصل أو **Backup**, **Managed يومي** (pgBackRest أو pg_dump) لـ Cloudflare R2 + اختبار استرجاع شهري.
- **Reverb** و **Ngix** على `wss://ws.domain`.
- الملفات على **Cloudflare R2**, والـ DNS و الـ CDN على Cloudflare.
- مراقبة: **Sentry** (Laravel + Nuxt + Flutter), Horizon dashboard, Uptime monitor, Telescope على الـ Staging بس.
- **Migrations** بـ zero-downtime: إضافة عمود ← نشر ← Backfill ← إزالة القديم في إصدار بعده.

18. خطة التنفيذ التفصيلية

الأسبوع 0 — التجهيز (أول يوم ويومين)

```
# 1) الـ Repo
mkdir -p apps packages infra docs

# 2) Laravel
composer create-project laravel/laravel apps/api
cd apps/api
php artisan install:api # Sanctum + routes/api.php
```

```

php artisan install:broadcasting # Reverb + Echo config
composer require laravel/horizon spatie/laravel-permission spatie/laravel-query-builder \
    spatie/laravel-activitylog brick/money propagandistas/laravel-phone dedoc/scramble
laravel/pennant
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan
laravel/pint
php artisan horizon:install
./vendor/bin/pest --init

# 3) Nuxt
cd ../ && pnpm dlx nuxi@latest init web && cd web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexie laravel-echo
pusher-js

# 4) Docker compose (pgsql, redis, mailpit, minio) + CI workflow + .editorconfig

```

- ☒ Postgres بدل SQLite في .env , APP_LOCALE=ar , APP_TIMEZONE=Africa/Cairo
- ☒ Pest + Larastan + Pint شغاليين في ال CI على أول PR.

الأسبوع 1-3 — الأساس (Identity + Catalog)

الأسبوع	المهام
1	Tenancy (scope + middleware + trait + تست العزل)، تسجيل محل جديد (Tenant + فرع + Seed + Owner)، Login، الفروع، Money cast، هيكل ال Domains
2	المستخدمين والأدوار والصلاحيات، الأجهزة و Layout، Audit log، PIN ال Auth، RTL + Nuxt اختيار الفرع
3	التصنيفات، الماركات، الموديلات، الأصناف والمتغيرات، التوافق، الباركود، بحث pg_trgm، استيراد أصناف من Excel

الأسبوع 4-8 — MVP البيع

الأسبوع	المهام
4	محرك المخزون (movements, levels, lots, FIFO, avg cost) + تستات ال Concurrency، الرصيد الافتتاحي
5	المشتريات والموردين وحساباتهم، مرتجع المشتريات
6	ال POS: شاشة البيع، البحث من Dexie، السلة، الخصومات، طرق الدفع، الفواتير المعلّقة، الطباعة 80mm
7	العملاء والآجل وكشف الحساب، المرتجعات من العملاء، الورديات والخزنة والمصروفات
8	التقارير الأساسية + لوحة المالك + (SaleCompleted, StockLow) Reverb + قوالب WhatsApp (الأساس)

الأسبوع 9-11 — الصيانة

- قوائم الأعطال + Checklist الاستلام + Seed افتراضي.
- استلام الجهاز، الإيصال ب QR، الحالات وال Timeline، القطع المسحوبة، العمولة، الضمان.
- قوالب رسائل الصيانة + شاشة الفنيين Real time + تقارير الأعطال.

الأسبوع 12-14 — الاستيراد ومرتجعات الموردين

- جهات الاستيراد، الشحنات، المصاريف وتوزيع التكلفة، الاستلام والنواقص.
- الدفعات وكشف الحساب.
- سلة المرتجعات، الفرز حسب المصدر، إذن المرتجع، الحالات والتأثير على الحسابات.

الأسبوع 15-17 — Billing

- الباقات والحدود و Pennant، التجربة، Paymob + Fawry + Manual، الـ Webhooks، الفواتير، الـ Scheduler والتذكيرات، Filament للإدارة.

الأسبوع 18-21 — Beta

- الأوفلاين الكامل (Outbox + Sync push/pull)، تحسين السرعة، 10-20 محل تجريبي، إصلاحات، Sentry، Backups، الـ Production.

بعد الإطلاق

- تطبيق المالك (Flutter)، المستعمل، الشحن والتحويلات، الفروع والتحويلات، ETA، QZ Tray، المتجر الأونلاين.

19. Definition of Done (لكل Feature)

- Migration قابلة للرجوع + Indexes.
- Model عليه BelongsToTenant + Policy + Casts.
- Form Request + Action + API Resource.
- تستات Feature (نجاح، Validation، صلاحيات، عزل المحلات).
- الشاشة في Nuxt بالعربي RTL وشغالة على شاشة الموبايل.
- Audit log للعمليات الحساسة.
- Pint + Larastan + Pest + Typecheck كلهم أخضر.
- اتجرب على الـ Staging.

20. أول 5 مهام تبدأ بيها النهارده

1. إنشاء الـ (Postgres/Redis) Docker compose + Nuxt + Laravel + Monorepo.
2. إعداد الـ (Pint + Larastan + Pest) CI من أول commit.

3. Tenant + Branch + User + trait الـ Tenancy + تست العزل.

4. تسجيل محل جديد (Onboarding API) مع Seed للأعطال والقوالب والأدوار.

5. Layout الـ Nuxt بالعربي Login + RTL + اختيار الفرع.